

Attention Networks

Yasser Hifny

Faculty of Computers and Artificial Intelligence
Capital University

Contents

- 1 Scaled Dot-Product Similarity Measure
- 2 Multi-head Self-Attention Networks
 - Numerical Example for Self-Attention
 - Masked Self-Attention
- 3 Stacking Self-Attention Layers
 - Position-wise Feed-Forward Network (FFN)
- 4 The Transformer Model
 - Neural Language Modeling
 - Conditional Language Modeling
- 5 Training and Inference for Encoder-Decoder Framework
 - Cross-Entropy Loss for Transformer Models
 - Inference for Transformer Models
- 6 Assignment

Scaled Dot-Product Similarity Measure

- Dot-product similarity is introduced first.
- Scaling by $\sqrt{d_k}$ is motivated by the growth of score variance with dimension.

Scaled Dot-Product Similarity Measure — Equations 1–3

$$\mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$$

$$\mathbf{q} = [0.8, 0.3, 0.5]$$

$$\mathbf{k}_1 = [0.9, 0.1, 0.4], \quad \mathbf{k}_2 = [0.4, 0.7, 0.5], \quad \mathbf{k}_3 = [0.7, 0.2, 0.6]$$

Scaled Dot-Product Similarity Measure — Equations 4–6

$$\mathbf{q} \cdot \mathbf{k}_1 = (0.8)(0.9) + (0.3)(0.1) + (0.5)(0.4) = 0.72 + 0.03 + 0.2 = 0.95$$

$$\mathbf{q} \cdot \mathbf{k}_2 = (0.8)(0.4) + (0.3)(0.7) + (0.5)(0.5) = 0.32 + 0.21 + 0.25 = 0.78$$

$$\mathbf{q} \cdot \mathbf{k}_3 = (0.8)(0.7) + (0.3)(0.2) + (0.5)(0.6) = 0.56 + 0.06 + 0.3 = 0.92$$

Scaled Dot-Product Similarity Measure — Equations 7–9

$$\frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d_k}} = \frac{\sum_{i=1}^{d_k} q_i k_i}{\sqrt{d_k}},$$

$$\mathbb{E}[q_i] = 0, \quad \mathbb{E}[k_i] = 0$$

$$\mathbb{E}[\mathbf{q} \cdot \mathbf{k}] = \mathbb{E} \left[\sum_{i=1}^{d_k} q_i k_i \right] = \sum_{i=1}^{d_k} \mathbb{E}[q_i k_i]$$

Scaled Dot-Product Similarity Measure — Equations 10–12

$$\mathbb{E}[q_i k_i] = \mathbb{E}[q_i] \mathbb{E}[k_i] = 0$$

$$\mathbb{E}[\mathbf{q} \cdot \mathbf{k}] = 0$$

$$\text{Var}(q_i k_i) = \mathbb{E}[(q_i k_i)^2] - (\mathbb{E}[q_i k_i])^2$$

Scaled Dot-Product Similarity Measure — Equations 13–15

$$\text{Var}(q_i k_i) = \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = \sigma^2 \sigma^2 = \sigma^4$$

$$\text{Var}(\mathbf{q} \cdot \mathbf{k}) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k \sigma^4$$

$$\text{Var}\left(\frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d_k}}\right) = \frac{d_k \sigma^4}{d_k} = \sigma^4$$

Multi-head Self-Attention Networks

- Self-attention transforms a sequence into contextual representations using query, key, and value projections.
- Multi-head attention combines several attention heads.

Multi-head Self-Attention Networks — Equations 16–18

$$Q = XW^Q \quad (n \times d_k), \quad K = XW^K \quad (n \times d_k), \quad V = XW^V \quad (n \times d_v)$$

$$A = QK^T \quad (n \times d_k) \times (d_k \times n) = (n \times n)$$

$$A = \frac{QK^T}{\sqrt{d_k}} \quad (n \times n)$$

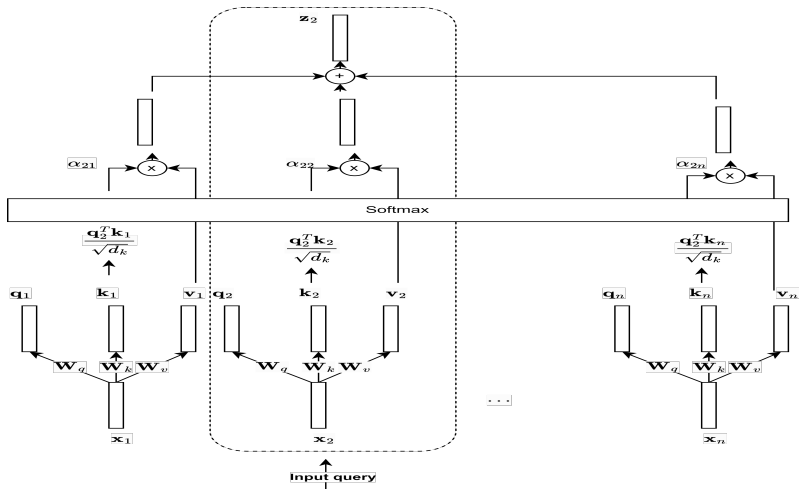
Multi-head Self-Attention Networks — Equations 19–21

$$Z = \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (n \times n) \times (n \times d_v) = (n \times d_v)$$

$$\alpha_{2i} = \frac{\exp(\mathbf{q}_2^\top \mathbf{k}_i)}{\sum_j \exp(\mathbf{q}_2^\top \mathbf{k}_j)},$$

$$\mathbf{z}_2 = \sum_i \alpha_{2i} \mathbf{v}_i,$$

Multi-head Self-Attention Networks — Figure 1



Multi-head Self-Attention Networks — Equations 22–24

$$Q_i = XW_i^Q \quad (n \times d_k), \quad K_i = XW_i^K \quad (n \times d_k), \quad V_i = XW_i^V \quad (n \times d_v)$$

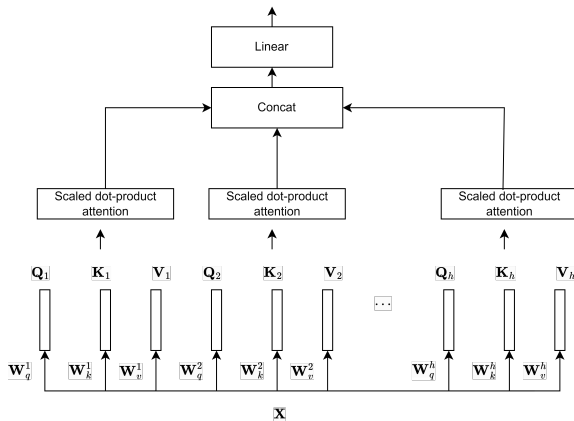
$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i) \quad (n \times d_v)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) \quad (n \times (h \cdot d_v))$$

Multi-head Self-Attention Networks — Equations 25–25

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h)W^O,$$

Multi-head Self-Attention Networks — Figure 2



Numerical Example for Self-Attention — Equations 26–28

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad (2 \times 2)$$

$$W^Q = W^K = W^V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad (2 \times 2)$$

$$Q = XW^Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad (2 \times 2)$$

Numerical Example for Self-Attention — Equations 29–31

$$K = XW^K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad (2 \times 2)$$

$$V = XW^V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad (2 \times 2)$$

$$A = QK^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad (2 \times 2)$$

Numerical Example for Self-Attention — Equations 32–34

$$A = \frac{QK^T}{\sqrt{2}} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.707 & 0 \\ 0 & 0.707 \end{bmatrix} \quad (2 \times 2)$$

$$\text{softmax}(A) = \begin{bmatrix} \frac{e^{0.707}}{e^{0.707} + e^0} & \frac{e^0}{e^{0.707} + e^0} \\ \frac{e^0}{e^{0.707} + e^0} & \frac{e^{0.707}}{e^{0.707} + e^0} \end{bmatrix} = \begin{bmatrix} 0.669 & 0.33 \\ 0.33 & 0.669 \end{bmatrix} \quad (2 \times 2)$$

$$\text{Attention}(Q, K, V) = \text{softmax}(A)V = \begin{bmatrix} 0.669 & 0.33 \\ 0.33 & 0.669 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 0.669 & 0.33 \\ 0.33 & 0.669 \end{bmatrix} \quad (2 \times 2)$$

Masked Self-Attention — Equations 35–37

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

$$A = \frac{QK^T}{\sqrt{d_k}}$$

$$A_{\text{masked}} = A + M$$

Masked Self-Attention — Equations 38–40

$$\text{Attention}(Q, K, V) = \text{softmax}(A_{\text{masked}}) V$$

$$A = \frac{QK^T}{\sqrt{2}} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$$M = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

Masked Self-Attention — Equations 41–42

$$A_{\text{masked}} = A + M = \begin{bmatrix} a_{11} & -\infty & -\infty \\ a_{21} & a_{22} & -\infty \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

$$\text{Attention}(Q, K, V) = \text{softmax}(A_{\text{masked}})V$$

Stacking Self-Attention Layers — Equations 43–45

Without intervening nonlinear transformations, stacking self-attention layers can collapse into a single linear transformation. The position-wise FFN as the nonlinear component was motivated.

$$Z_1 = \text{Attention}(Q_1, K_1, V_1) = \text{softmax} \left(\frac{Q_1 K_1^\top}{\sqrt{d_k}} \right) V_1$$

$$Q_1 = XW_1^Q, \quad K_1 = XW_1^K, \quad V_1 = XW_1^V$$

$$\begin{aligned} Z_1 &= \text{softmax} \left(\frac{(XW_1^Q)(XW_1^K)^\top}{\sqrt{d_k}} \right) (XW_1^V) \\ &= AXW_1^V \end{aligned}$$

Stacking Self-Attention Layers — Equations 46–48

$$Z_2 = \text{Attention}(Q_2, K_2, V_2) = \text{softmax} \left(\frac{Q_2 K_2^\top}{\sqrt{d}} \right) V_2$$

$$Q_2 = Z_1 W_2^Q, \quad K_2 = Z_1 W_2^K, \quad V_2 = Z_1 W_2^V$$

$$Q_2 = A X W_1^V W_2^Q$$

Stacking Self-Attention Layers — Equations 49–51

$$K_2 = AXW_1^V W_2^K$$

$$V_2 = AXW_1^V W_2^V$$

$$W^Q = W_1^V W_2^Q, \quad W^K = W_1^V W_2^K, \quad W^V = W_1^V W_2^V$$

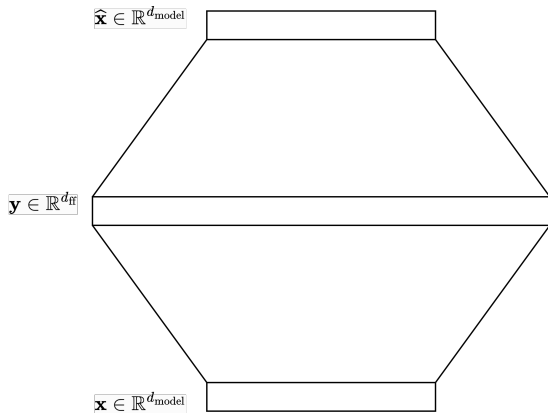
Stacking Self-Attention Layers — Equations 52–52

$$\begin{aligned} Z_2 &= \text{Softmax} \left(\frac{(AXW_1^V W_2^Q)(AXW_1^V W_2^K)^\top}{\sqrt{d_k}} \right) AXW_1^V W_2^V \\ &= \text{Softmax} \left(\frac{(AXW^Q)(AXW^K)^\top}{\sqrt{d_k}} \right) AXW^V \end{aligned}$$

Position-wise Feed-Forward Network (FFN) — Equations 53–53

$$\mathbf{y} = \mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2,$$

Position-wise Feed-Forward Network (FFN) — Figure 3



The Transformer Model

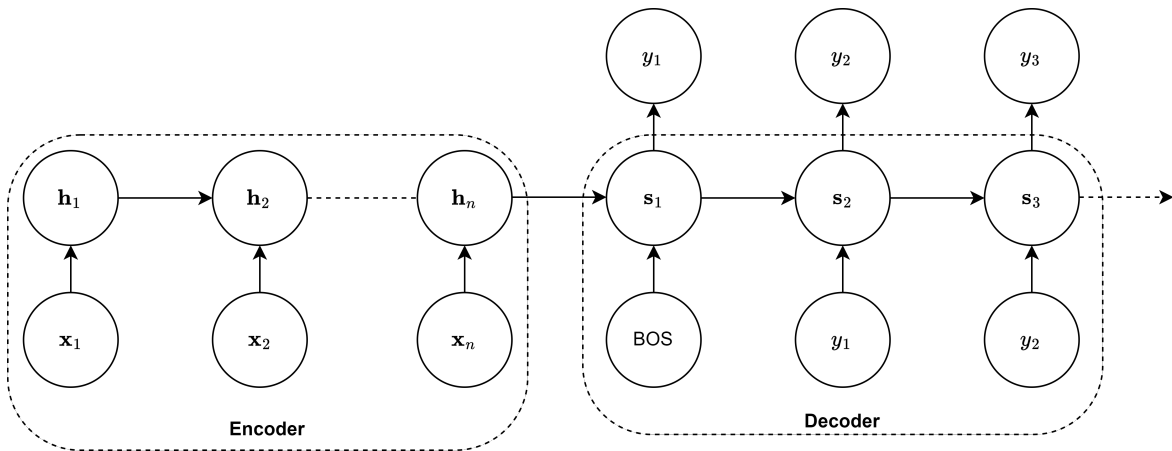
- The Transformer combines attention, feed-forward layers, residual connections, normalization, and positional information.

Conditional Language Modeling — Equations 81–82

$$P(Y \mid X) = P(y_1, y_2, \dots, y_n \mid x_1, x_2, \dots, x_m)$$

$$P(Y \mid X) = \prod_{t=1}^n P(y_t \mid y_1, y_2, \dots, y_{t-1}, X)$$

Conditional Language Modeling — Figure 4



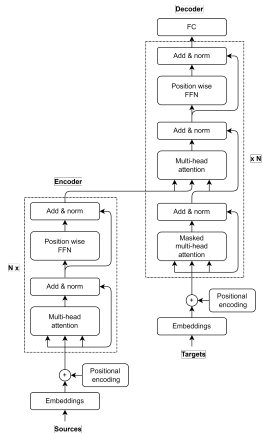
Conditional Language Modeling — Equations 83–85

$$\mathbf{H}^X = \text{Encoder}(\mathbf{E}^X)$$

$$\mathbf{H}_t^Y = \text{Decoder}(\mathbf{E}_{<t}^Y, \mathbf{H}^X)$$

$$P(y_t \mid y_1, y_2, \dots, y_{t-1}, X) = \text{softmax}(\mathbf{W}_o \mathbf{H}_t^Y)$$

Conditional Language Modeling — Figure 5

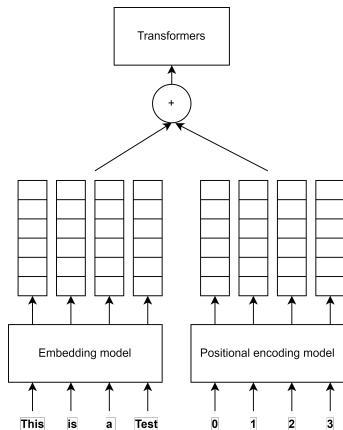


Conditional Language Modeling — Equations 86–87

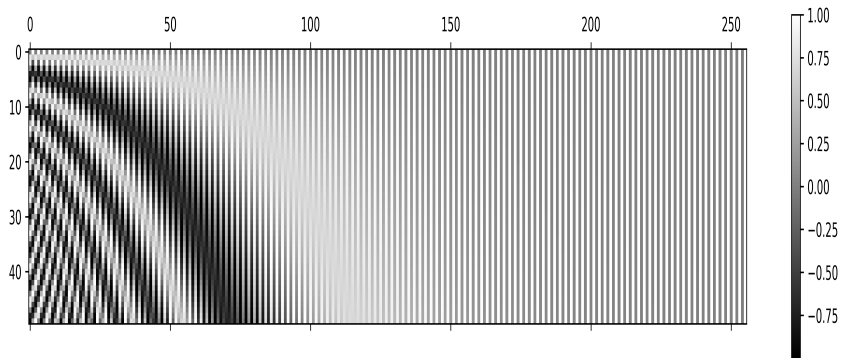
$$\text{PE}_{(\text{pos}, 2i)} = \sin \left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}} \right)$$

$$\text{PE}_{(\text{pos}, 2i+1)} = \cos \left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}} \right),$$

Conditional Language Modeling — Figure 6



Conditional Language Modeling — Figure 7



Conditional Language Modeling — Equations 88–90

Input Embedding + Positional Encoding

for each layer l :

$$\hat{X}^l = \text{LayerNorm}(X^{l-1} + \text{MultiHead}(X^{l-1}, X^{l-1}, X^{l-1}))$$

$$X^l = \text{LayerNorm}(\hat{X}^l + \text{FFN}(\hat{X}^l))$$

Output of Encoder = X^N

Input Embedding + Positional Encoding

for each layer l :

$$\hat{Y}^l = \text{LayerNorm}(Y^{l-1} + \text{MaskedMultiHead}(Y^{l-1}, Y^{l-1}, Y^{l-1}))$$

$$\tilde{Y}^l = \text{LayerNorm}(\hat{Y}^l + \text{MultiHead}(\hat{Y}^l, X^N, X^N))$$

$$Y^l = \text{LayerNorm}(\tilde{Y}^l + \text{FFN}(\tilde{Y}^l))$$

Output of Decoder = Y^N

$$\text{Output Probabilities} = \text{softmax}(Y^N W_{\text{final}} + b_{\text{final}})$$

Layer Norm — Equations 91–93

LayerNorm($X + \text{SubLayer}(X)$),

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i$$

Layer Norm — Equations 94–96

$$\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$$

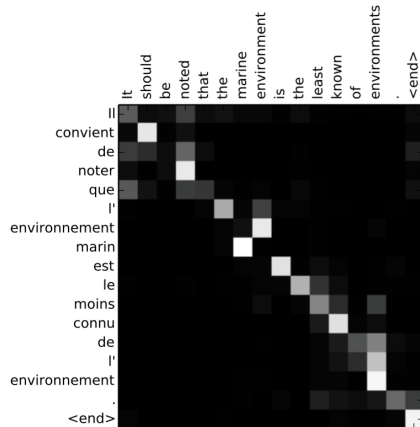
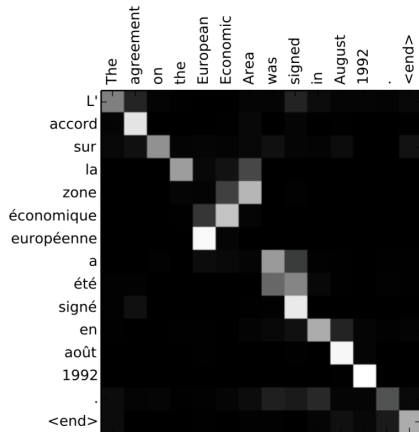
$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$y_i = \gamma \hat{x}_i + \beta$$

Cross Attention — Equations 97–97

$$\text{MultiHead}(\hat{Y}', X^N, X^N),$$

Cross Attention — Figure 8



Cross-Entropy Loss for Transformer Models — Equations 98–98

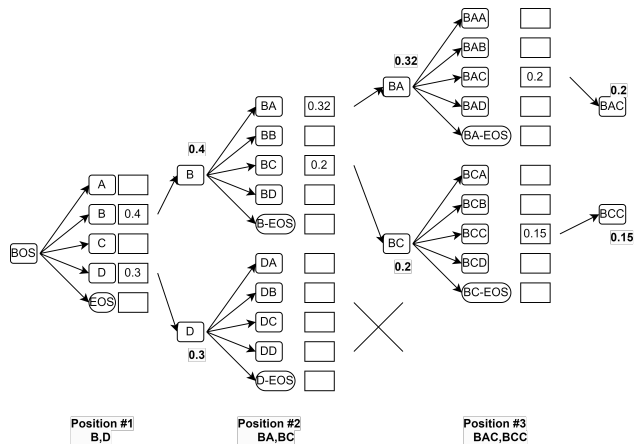
$$E = - \sum_{i=1}^n \log P(y_i \mid X, y_{1:i-1}),$$

Inference for Transformer Models — Equations 99–100

$$y_i = \arg \max_{y_i} P(y_i \mid y_1, y_2, \dots, y_{i-1}, X)$$

$$S^{(i)} = \arg \max_{S^{(i)}} \sum_{j=1}^i \log P(y_j^{(i)} \mid y_1^{(i)}, y_2^{(i)}, \dots, y_{j-1}^{(i)}, X)$$

Inference for Transformer Models — Figure 9



Assignment

- Implement a neural machine translation algorithm using the Transformer model using the Keras deep learning library and Google colab. To load the dataset, please visit https://github.com/SamirMoustafa/nmt-with-attention-for-ar-to-en/blob/master/ara_.txt.

Thank You

Questions?